Assessment Brief

Formative Assessment Identification

Programme	BSc (Hons) Software Engineering
Module title	Introduction to Programming and Databases
Academic level / year	1
Teaching period	August to October 2026
Assessment title	Peer Learning Project: Python Menu-Driven CLI Application
Faculty	Chris Bergue

1. Assessment Purpose and Context

This formative Peer Learning Project evaluates your ability to plan a small software solution and implement a working Python command-line, menu-driven application aligned to a real-world / GCGO-related problem.

At this stage of the module, you are expected to apply the programming foundations covered so far: variables, control flow, functions, lists and dictionaries, basic algorithms / data-structure thinking (DSA), input validation, and file handling. Build a clear procedural Python programme organised into modules and functions. Keep the design simple and readable so you can explain every part of your code confidently in your demonstration.

2. Learning Outcomes Assessed

Learning outcome	How the assessment provides evidence
Knowledge and Understanding	Shows understanding of Python fundamentals (control flow, functions, lists/dictionaries), basic data processing, file handling, and modular script organisation.
Practice: Applied Knowledge and Understanding	Builds a working command-line application that solves a scoped problem using menus, validation, and file-based storage.
Generic Cognitive Skills	Analyses a problem domain, defines requirements, designs a simple data model using lists/dicts, and handles runtime errors carefully.
Communication, ICT, and Numeracy Skills	Documents the solution via a short requirements note, inline comments, README usage instructions, and a video demonstration.
Autonomy, Responsibility, and Working with Others	Takes ownership of planning and implementation decisions and submits a complete, well-organised project package.

3. Assessment Task

Design, document, and build a modular menu-driven Python application that helps users track, process, and analyse data for a chosen real-world problem.

You must choose ONE of the following three scenarios for your application:

- **Campus Resource Booking Helper:** Track bookable campus resources (rooms, equipment, study pods), booking slots, user requests, cancellations, and simple availability reports.
- **Community Outreach Activity Logger:** Track outreach activities linked to a GCGO theme, volunteer contributions, hours logged, locations, and impact summaries.
- **Personal Budget & Savings Goal Tracker:** Track income/expense entries by category, savings goals, progress percentages, and monthly summary reports.

In your short requirements note, briefly explain how the chosen scenario connects to at least one Global Challenge / mission interest (GCGO).

4. Required Deliverables

Submit a complete digital package. Required deliverables:

- **Public GitHub Repository:** Clean, well-commented source code in the required modular structure, commit history, sample data file(s), and README.
- **Short Requirements Note (1–2 pages PDF or Markdown in repo):** Problem statement, chosen scenario, functional requirements list, and data fields stored in your JSON/CSV file(s).
- **Working Python CLI Application:** Menu-driven prototype with create/view/update/delete (or equivalent), search/filter, and at least two analysis/report features; data must persist between runs via JSON or CSV.
- **Demo Walkthrough (10–15 minutes):** Feature demo of all menu operations and persistence, plus a code walkthrough of modular separation, validation, and file I/O.
- **Sources & AI Disclosure (if applicable):** Repository link, any reused snippets with attribution, and AI usage disclosure (APA 7th where relevant).

5. Task Requirements and Expected Structure

Application Structure

Organise your project into clear modules and functions. Recommended minimum structure:

`main.py` — menu loop and user interaction

`validation.py` — input checks and sanitisation helpers

`data_store.py` — load/save JSON or CSV using the standard library

`processing.py` — search, filter, totals, reports, and other processing logic

`data/` — sample dataset file(s) used by the app

`README.md` — how to run the app and overview of features

Technical Requirements

1. Menu-Driven Command-Line Interface

- **Persistent menu loop:** Present numbered options until the user chooses Exit.
- **Clear prompts:** Each action must guide the user and confirm success/failure.
- **Navigation:** Invalid menu choices must be handled without crashing.

2. Data Model using Built-in Structures

- **Lists and dictionaries:** Represent each record as a dictionary and collections of records as lists (or nested structures where useful).
- **Functions and modules:** Break logic into reusable functions across files; keep your code easy to follow.

- **DSA thinking:** Use loops, searches, filters, counting, and simple aggregations to process your data.

3. File Persistence (JSON or CSV)

- **Save and reload:** Application data must survive restart by reading/writing a data file.
- **Standard library:** Use Python's json and/or csv modules.
- **File errors:** Handle missing or unreadable files gracefully with try/except and clear messages.

4. Core Features

Your application must include all of the following:

- **Add / Create records** with validated fields
- **View / List records** in a readable format
- **Update and Delete** (or archive) existing records
- **Search or Filter** by at least one meaningful field
- **Two analysis features** (examples: totals, averages, counts by category, progress %, top items, or a simple text report)

5. Validation and Defensive Programming

- **Input validation:** Reject empty required fields; validate numeric ranges and simple formats where relevant.
- **Exception handling:** Use try/except around file I/O and risky conversions (for example int/float parsing).
- **Code comments:** Comment each major function explaining purpose and key logic.

6. Scope for this Formative

This assignment focuses on the skills taught so far in the module. Please stay within that scope:

Use / focus on	Out of scope for now
Python 3 standard library	Database systems and database libraries (these come later in the module)
Functions, modules, lists, dicts, sets	Object-oriented class design (introduced later)
JSON and/or CSV file storage	Full web/GUI frameworks as the main solution
Procedural menu-driven CLI	Over-engineered designs you cannot explain clearly

6. Process, Milestones and Checkpoints

Stage	Required evidence or activity	Purpose
Stage 1: Ideation & Requirements	Choose one scenario; write a short requirements note (problem, users, features, data fields, GCGO link).	Ensures the project scope is clear and achievable.
Stage 2: CLI Prototype & Persistence	Implement the menu loop, core record operations, validation, and JSON/CSV save-load.	Confirms core Python skills and file handling before analysis features.
Stage 3: Analysis Features & Hardening	Add search/filter + two analysis/report features; improve error handling and README.	Demonstrates data processing and careful programming practice.
Stage 4: Demo Video & Submission	Record an 8–12 minute walkthrough; push your public GitHub repo; submit as instructed on Canvas.	Final check of the working application and project ownership.

7. Use of Generative AI and Digital Tools

AI tools (ChatGPT, Gemini, Copilot, etc.) may be used as learning aids—for example to explain errors, discuss ideas, or improve wording in documentation.

- **Unacceptable use:** Submitting an AI-generated whole application, or substantial code you cannot explain in your demo.
- **Disclosure:** If AI helped write or fix non-trivial code, log it in your Sources & AI Disclosure document.

8. Academic Integrity and Referencing

All submitted Python code must be your own original work for this formative.

- **Code attribution:** Acknowledge adapted snippets with inline comments and list URLs in Sources.
- **Plagiarism:** Copying from peers, online repositories, or previous assignments is prohibited under institutional regulations.

9. Marking Criteria and Rubric

Criterion	Weight	Unsatisfactory	Developing	Good	Very Good	Exceptional
Problem framing & requirements note	10	1–2: Missing/unclear problem; no useful requirements.	3–4: Weak scope; limited feature list.	5–6: Clear scenario and basic requirements.	7–8: Well-scoped requirements with data fields and GCGO link.	9–10: Excellent, realistic requirements and clear success criteria.
Menu-driven CLI usability	15	1–3: Broken menu; frequent crashes.	4–6: Basic menu with navigation issues.	7–9: Usable menu covering core actions.	10–12: Clear prompts; robust invalid-option handling.	13–15: Polished flow; consistent confirmations and exit paths.
File persistence (JSON/CSV)	15	1–3: No persistence or data is lost on restart.	4–6: Partial save/load with bugs.	7–9: Works for normal cases.	10–12: Reliable reload after restart; sensible file layout.	13–15: Robust I/O with strong missing/unreadable-file handling.
Core features (CRUD + search)	15	1–3: Major features missing.	4–6: Only create/view partially working.	7–9: Create/view/update/delete mostly complete.	10–12: Full record operations + working search/filter.	13–15: Complete feature set with clean record handling.
Analysis / reporting features	15	1–3: No analysis features.	4–6: One weak/incorrect report.	7–9: Two basic analyses present.	10–12: Two solid, accurate analyses.	13–15: Clear, useful analyses with well-presented output.
Validation & defensive coding	15	1–3: Little validation; crashes on bad input.	4–6: Minimal checks; uneven try/except.	7–9: Key fields validated; basic exception handling.	10–12: Consistent validation and safe conversions.	13–15: Thorough checks and resilient error paths.

Code quality, modularity & comments	10	1–2: Hard to follow; poor structure.	3–4: Weak structure; sparse comments.	5–6: Modular functions; adequate comments.	7–8: Clean modules; good naming; clear comments.	9–10: Excellent, explainable procedural design.
Demo & repository submission	5	1: Missing repo/demo.	2: Incomplete demo or weak README.	3: Basic demo and repo present.	4: Clear demo covering features + code.	5: Strong 8–12 min demo; complete public repo/history.
Total	100					

10. Feedback and Use of Feedback

Feedback will be provided via Canvas within institutional marking timelines. Use the rubric comments to strengthen your procedural design, validation, and file-handling practice as the module continues.

11. Student Submission Checklist

☐ I chose one of the three approved scenarios and linked it to a GCGO/mission interest.

☐ My app is a menu-driven Python CLI with JSON or CSV persistence.

☐ I organised my code into modules and functions I can explain.

☐ I implemented create/view/update/delete (or equivalent), search/filter, and two analysis features.

☐ I included validation and try/except around risky operations.

☐ My GitHub repo contains source files, sample data, README, and commit history.

☐ I recorded an 8–12 minute demo and code walkthrough.

☐ I disclosed AI assistance if used.

12. Additional Resources

Resource	Purpose	Link / Location
Menu-driven Python programme examples	Reference patterns for CLI menus	Link
Python json module docs	Saving and loading structured data in files	Link
Python csv module docs	Working with tabular data files	Link
Software Requirements Specification format	Stage 1 requirements note structure	Link
Python coding best practices	Naming, functions, readability	Link
Generative AI guidance	Responsible AI use and disclosure	Link

13. Questions and Clarification

Post clarification questions in the Canvas Discussion Board for this module so all students can benefit from the same answers.

For private queries about your code, contact jbergue@alueducation.com or use scheduled coaching sessions.